A dissertation submitted to the University of Greenwich in partial fulfilment of the requirements for the Degree of

Master of Science

in

Data Science

Database of Fire Safety Regulations with Version Comparison Using ML and NLP

Clause-Level Change Detection Across Editions and Jurisdictions

Name: Praveen Govindaraj

Student ID: 001492439

Supervisor: Chan Sorayudh Chanthan

Submission Date: September 7, 2026

Target Word Count: 14,000 words

School of Computing and Mathematical Sciences
University of Greenwich, London, UK

Abstract

Fire safety regulations in the built environment undergo rapid, continuous revisions triggered by technical field evaluations, statutory updates, and public inquiries such as the Phase 2 Grenfell Tower Inquiry. In the United Kingdom, statutory guidance such as *Approved Document B* (ADB) has expanded significantly. Between 2019 and 2025, England's Approved Document B expanded by 22.7% in total word count, with the 2022 to 2025 revision alone altering 38% of its statutory clauses and introducing 11,220 words of new technical obligations. Manually tracking and verifying these revisions across unstructured, multi-column Portable Document Format (PDF) files is labour-intensive, computationally unstructured, and prone to human oversight. Traditional string-based diff utilities are fundamentally inadequate for this task; they generate excessive syntactic noise when non-substantive stylistic rewrites occur, while simultaneously failing to flag critical numerical shifts (e.g., residential sprinkler thresholds decreasing from 30 m to 11 m) due to high character overlap.

To address these domain-specific vulnerabilities, this dissertation presents the design, implementation, and empirical validation of an automated regulatory database and clause-level comparison platform utilizing Natural Language Processing (NLP) and Machine Learning (ML). The ingestion pipeline processes raw, unstructured regulatory PDF texts using coordinate-aware block sorting via PyMuPDF, maps provisions into a normalized Third Normal Form (3NF) relational SQLite schema managed via SQLAlchemy, and generates semantic vector representations using bi-encoder Sentence Transformers (`BAAI/bge-base-en-v1.5` and `sentence-transformers/all-MiniLM-L6-v2`). To resolve the critical failure mode wherein dense transformer embeddings remain insensitive to numerical and modal modifications (producing cosine similarities exceeding 0.99 despite major regulatory shifts), this research develops a deterministic Word-Evidence Guard. This defensive mechanism intercepts embeddings and forces reclassification whenever numerical thresholds, dimensional units, or statutory modal operators differ between editions.

The production platform indexes 25 editions and 10,697 clauses across five jurisdictions—including 9,050 core UK and Irish statutory provisions and 1,647 international comparative baseline clauses—within a full-stack architecture built with FastAPI, Next.js, and TypeScript. Evaluated against official government amendment registers issued by HM Government, the platform achieved 100% recall (8/8 amendments surfaced on the 2025 register) and attained an alignment F1-score of 0.856 when clause identifiers were completely masked. In practice, the platform reduces the auditor reading burden by 70.0%, allowing building control officers and fire engineers to review 150 flagged clauses rather than manually auditing 492 full sections.

Keywords: Fire Safety Regulations, Natural Language Processing, Sentence Transformers, Regulatory Compliance, Clause Alignment, Word-Level Redline, Word-Evidence Guard, Harvard Referencing.

Preface

This dissertation is submitted in partial fulfilment of the requirements for the Master of Science in Data Science at the University of Greenwich. The work builds directly upon core competencies developed in **COMP-1801 (Machine Learning)** and **COMP-1832 (Programming Fundamentals for Data Science)**.

The research addresses the practical challenge of managing rapid regulatory updates in the built environment. In alignment with Level 7 National Qualifications Framework (NQF) descriptors, this dissertation demonstrates independent problem formulation, end-to-end data pipeline engineering, empirical benchmarking against published government ground truth, and a critical analysis of where dense vector embeddings fail in safety-critical legal domains.

Acknowledgements

I would like to express my sincere gratitude to my academic supervisor, Chan Sorayudh Chanthan, for his continuous guidance, technical advice, and constructive feedback throughout the course of this project. His suggestions during our progress meetings helped refine the hybrid comparison logic and improve the experimental evaluation.

I would also like to thank the second marker and the academic staff in the School of Computing and Mathematical Sciences for providing the computing resources, software environments, and module support that made this dissertation possible.

Contents

1 Introduction **1**

1.1 Overview and Problem Domain Discussion 1

1.2 Project Objectives 2

1.2.1 Research Questions 3

1.3 Report Structure 3

2 Literature Review & Background Research **5**

2.1 Literature Review 5

2.1.1 Information Extraction and Parsing from Complex Built-Environment Codes 5

2.1.2 Text Representation: Lexical Metrics vs. Dense Semantic Vector Embeddings 6

2.1.3 Vulnerability of Dense Encoders to Numerical and Modal Perturbations 6

2.1.4 Machine Learning and Automated Compliance in the Built Environment 7

2.2 Background Research 7

2.2.1 Primary Research: Statutory Document Inspection 7

2.2.2 Comparative Analysis of State-of-the-Art Approaches 8

2.2.3 Summary 8

3 Requirements Analysis **9**

3.1 User Personas and Stakeholder Analysis 9

3.2 Functional Requirements 9

3.3 Non-Functional Requirements 10

3.4 Data Scope and Corpus Boundaries 11

4 Methodology **12**

4.1 Methodological Framework 12

4.2 System Architecture 12

4.3 Formal Comparison and Classification Formulation 13

4.3.1 Intra-Jurisdictional Alignment 13

4.3.2 Cross-Jurisdictional Bipartite Matching Formulation 13

4.3.3 Classification Decision Boundaries . . . 14
4.4 Word-Evidence Guard Logic . . . 14
4.5 Legal, Social, Ethical and Professional Issues (LSEPI) . . . 15
4.5.1 Legal Considerations and Intellectual Property . . . 15
4.5.2 Ethical Implications in Life-Safety Domains . . . 15
4.5.3 Social Impact and Industry Accessibility . . . 15
4.5.4 Professional Standards and Engineering Integrity . . . 16

5 Designing the Solution 17
5.1 PDF Ingestion Pipeline Design . . . 17
5.2 Database Schema Design . . . 18

6 Implementation of the System 20
6.1 Hybrid Comparison Engine . . . 20
6.2 Full-Stack Web Interface . . . 21
6.2.1 Dashboard Architecture and Global Controls . . . 22
6.2.2 Side-by-Side Visual Redline . . . 22

7 Testing the Solution 24
7.1 Testing Strategy . . . 24
7.2 Validation Against Ground Truth Registers . . . 24
7.3 Parsing Accuracy Verification . . . 25

8 Investigation and Analysis of the Results 26
8.1 Accuracy Benchmark Against Official Registers (RQ1) . . . 26
8.2 Patterns of Regulatory Evolution (RQ2) . . . 26
8.3 Cross-Border Alignment and Search Usability (RQ3) . . . 27
8.4 Encoder Failure Analysis and Guard Verification (RQ4) . . . 28
8.5 Auditor Workload Reduction . . . 28

9 Evaluation of the Designs 29
9.1 Architectural and Design Review . . . 29

10 Critical Evaluation 30
10.1 Overview and Summary of Achievements . . . 30
10.2 Evaluation of Results Against Objectives . . . 30
10.3 Recommendations and Areas for Future Work . . . 30
10.4 Personal Evaluation of the Project Work . . . 31

A Regulatory Corpus Metadata 34

List of Figures

4.1 Four-Tier System Architecture for Regulatory Version Comparison. 12

5.1 Entity-Relationship Diagram of the Relational Clause Database. 19

6.1 Overview of the Regulation Diff Web Dashboard showing edition selection, comparison model settings, and summary change statistics. 22

6.2 Synchronized dual-pane redline display highlighting additions in green, deletions in red strikethrough, and the Word-Evidence Guard override badge. 23

8.1 Distribution of Clause Changes in ADB 2019 vs 2025 Revision (617 Total Clauses). 27

8.2 Full-text keyword filtering within an active comparison, isolating clauses referencing escape routes and exit corridors. 27

List of Tables

2.1 Comparison of Document Comparison Paradigms and the Proposed System . . 8

8.1 System Performance Metrics on Statutory Benchmark 26

A.1 Summary of Ingested Regulatory Documents across Jurisdictions 34

List of Acronyms

3NF	Third Normal Form
ADB	Approved Document B (Building Regulations England and Wales)
API	Application Programming Interface
BCS	British Computer Society
BERT	Bidirectional Encoder Representations from Transformers
BGE	BAAI General Embedding
BIM	Building Information Modeling
CAD	Computer-Aided Design
DLUHC	Department for Levelling Up, Housing and Communities
ERD	Entity-Relationship Diagram
FSO	Regulatory Reform (Fire Safety) Order 2005
GDPR	General Data Protection Regulation
GPU	Graphics Processing Unit
IFC	Industry Foundation Classes
JSON	JavaScript Object Notation
LCS	Longest Common Subsequence
LLM	Large Language Model
LSEPI	Legal, Social, Ethical, and Professional Issues
NER	Named Entity Recognition
NLP	Natural Language Processing
NQF	National Qualifications Framework
OGL	Open Government Licence
ORM	Object-Relational Mapping
PDF	Portable Document Format
REST	Representational State Transfer
RAG	Retrieval-Augmented Generation
SBERT	Sentence-BERT
TGD-B	Technical Guidance Document B (Republic of Ireland)
UI	User Interface
URI	Uniform Resource Identifier

Chapter 1

Introduction

1.1 Overview and Problem Domain Discussion

Building regulations and statutory fire safety standards establish mandatory technical thresholds designed to safeguard human life, facilitate emergency response operations, and mitigate structural damage in the built environment. In the United Kingdom and neighbouring jurisdictions, the statutory fire safety framework is distributed across primary legislation, secondary statutory instruments, and non-mandatory technical guidance books that serve as standard routes to legal compliance. In England and Wales, statutory guidance is published through *Approved Document B* (ADB) (HM Government, 2019); in Scotland through the *Building Standards Technical Handbooks* (Section 2: Fire) (Li et al., 2024); in Northern Ireland through *Technical Booklet E*; and in the Republic of Ireland through *Technical Guidance Document B* (TGD-B).

Historically, these regulatory frameworks operated on multi-year revision timelines, with major overhauls occurring approximately once every decade. However, the catastrophic fire at Grenfell Tower in June 2017—and the subsequent findings of the Hackitt Review and the Phase 2 Inquiry—prompted a fundamental restructuring of regulatory oversight in the United Kingdom. Regulators adopted a continuous revision strategy. In England alone, the Department for Levelling Up, Housing and Communities (DLUHC) published successive amendment booklets, circulars, and transitional updates in 2019, 2020, 2022, 2024, and 2025. Over this six-year period, England's Approved Document B expanded by 22.7% in total word volume. The single revision cycle spanning 2022 to 2025 altered 38% of its technical provisions, adding 11,220 words of new obligations concerning combustible materials, evacuation lifts, external cladding limitations, and multi-staircase requirements.

A fundamental operational bottleneck for fire engineers, local authority building control officers, structural designers, and corporate auditors is that statutory building codes are disseminated almost exclusively as static, unstructured Portable Document Format (PDF) files. Government authorities provide neither centralized machine-readable databases nor structured digital changelogs highlighting semantic modifications. Consequently, industry practitioners are

forced to conduct manual reviews across hundreds of dense pages to isolate newly enforced rules (Li et al., 2024).

Standard text-comparison algorithms and diff software packages (such as GNU Diff, Git word-diff, or standard Myers-difference utilities) fail when applied to this domain due to two fundamental architectural issues:

1. **Syntactic Sensitivity and Semantic Blindness:** When legislative drafting committees alter the phrasing of a clause to improve clarity or align with modern drafting guidelines without changing its legal obligation, lexical diff tools mark the entire text block as modified. This generates excessive false-positive noise, obscuring substantive changes.

2. **Insensitivity to Critical Numerical Shifts:** When a critical numerical threshold is revised—such as lowering the trigger height for mandatory residential sprinkler installations from 30 m to 11 m, or lowering the requirement for a second escape stair from 30 m to 18 m—character-overlap utilities record only a fractional string variance. These tools treat a life-critical technical modification with the same severity as the correction of a typographical error.

1.2 Project Objectives

The central goal of this research project is to design, develop, and evaluate an automated, clause-level regulatory database and comparison platform that parses unstructured PDF fire codes, indexes them into a structured relational schema, and highlights substantive changes across editions and jurisdictions.

To achieve this goal, the project was structured around six SMART (Specific, Measurable, Achievable, Relevant, and Time-bound) research objectives:

- **Objective 1 (Corpus Ingestion):** Develop a coordinate-aware layout extraction pipeline to parse multi-column PDF publications across England & Wales, Scotland, Northern Ireland, and the Republic of Ireland, extracting over 10,000 clauses into clean data structures.

- **Objective 2 (Methodological and Literature Review):** Conduct a systematic survey of layout-aware parsing methods, dense bi-encoder sentence embeddings (`BGE`, `SBERT`), and lexical similarity measures across legal and engineering domains (Reimers and Gurevych, 2019; Li et al., 2024; Wang et al., 2023).

- **Objective 3 (Database Engineering):** Design and implement a normalized Third Normal Form (3NF) relational database in SQLite via SQLAlchemy ORM to manage documents, temporal versions, atomic clauses, and vector embeddings.

- **Objective 4 (Hybrid Engine with Word-Evidence Guard):** Implement a multi-stage comparison engine combining exact clause-identifier matching, dense semantic cosine

similarity, and a deterministic Word-Evidence Guard that detects modified numbers, measurement units, and statutory modal operators.

- **Objective 5 (Interactive Full-Stack Web Interface):** Build and containerize a web application using FastAPI, Next.js, and TypeScript to provide auditors with synchronized redline displays, document diff spines, and structured export tools.
- **Objective 6 (Empirical Evaluation):** Benchmark the platform against official government amendment registers to evaluate statutory recall, alignment F1-scores, and auditor workload reduction.

1.2.1 Research Questions

This investigation is guided by four distinct research questions:

- **RQ1:** To what degree of accuracy and recall can an automated NLP pipeline align revised clauses across successive editions when evaluated against published government amendment registers?
- **RQ2:** What longitudinal patterns (e.g., word count growth, clause replacement versus insertion, section-level volatility) describe the historical evolution of fire safety guidance over successive revision cycles?
- **RQ3:** Can dense semantic vector embeddings successfully align equivalent fire safety provisions across differing national jurisdictions that share no common numbering convention?
- **RQ4:** Under what conditions do dense sentence transformers fail to flag safety-critical numerical changes, and how effectively can deterministic lexical guards eliminate these false-negative errors?

1.3 Report Structure

The remainder of this dissertation is organized as follows:

- **Chapter 2 (Literature Review & Background Research):** Reviews regulatory document extraction, dense embedding theory, and the limitations of deep learning models in numerical and compliance contexts.
- **Chapter 3 (Requirements Analysis):** Details user personas, functional (FR1–FR8) and non-functional (NFR1–NFR6) requirements, and corpus boundaries.

- **Chapter 4 (Methodology):** Presents the formal mathematical formulations of the comparison engine, the Word-Evidence Guard logic, and a review of Legal, Social, Ethical, and Professional Issues (LSEPI).
- **Chapter 5 (Designing the Solution):** Outlines the layout-aware extraction heuristics, block-sorting rules, and the 3NF relational database schema.
- **Chapter 6 (Implementation of the System):** Covers the core implementation of the comparison engine, API services, and the full-stack user interface.
- **Chapter 7 (Testing the Solution):** Details the testing strategy across unit, integration, and regression suites, including parser boundary validation.
- **Chapter 8 (Investigation and Analysis of the Results):** Provides empirical analyses addressing RQ1 through RQ4 and quantifies auditor workload reduction.
- **Chapter 9 (Evaluation of the Designs):** Evaluates the system's architectural modularity, database performance, and operational constraints.
- **Chapter 10 (Critical Evaluation):** Reflects on project achievements against objectives, details technical limitations, and outlines directions for future work.

Chapter 2

Literature Review & Background Research

2.1 Literature Review

Applying automated natural language processing and information retrieval techniques to statutory text introduces unique engineering challenges. Unlike narrative prose, news articles, or colloquial conversations, legal and regulatory documents are characterized by strict hierarchical structures, cross-referential syntax, precise technical terminology, and prescriptive requirements (Li et al., 2024; Elhosary and Moselhi, 2025).

2.1.1 Information Extraction and Parsing from Complex Built-Environment Codes

Extracting machine-readable text from regulatory PDF publications is difficult due to non-standard layout typography. As observed by Li et al. (2024), building standards rely heavily on multi-column text formatting, interspersed technical diagrams, callout boxes, and tabular provisions. Traditional text extraction tools that parse character streams based solely on their internal byte order frequently interleave adjacent columns. This creates text bleeding, where a paragraph from column one merges into column two, corrupting the semantic integrity of the extracted clauses.

Recent work has explored deep learning approaches for visual document understanding, such as LayoutLM, to classify visual tokens based on both spatial coordinates and linguistic features. However, Li et al. (2024) found that when applied to prescriptive regulatory documents, supervised visual transformer models often struggle with fine-grained edge cases, such as alphanumeric sub-clause hierarchies (e.g., Clause 3.12(b)(iv)) or continuous margins. Rule-based, coordinate-aware bounding-box sorting algorithms implemented via low-level PDF parsing engines (such as PyMuPDF) provide higher determinism and precision for statutory texts by preserving block sorting order without requiring large labelled training sets.

2.1.2 Text Representation: Lexical Metrics vs. Dense Semantic Vector Embeddings

Automated document versioning historically relied on string-distance metrics derived from dynamic programming. Levenshtein Distance measures the minimum number of single-character edits (insertions, deletions, substitutions) required to transform one string into another. The Longest Common Subsequence (LCS) metric focuses on identifying the longest shared sequence of tokens, forming the mathematical foundation of standard Unix diff utilities. While these algorithms are computationally efficient, they are completely insensitive to semantic context. If an administrative body replaces the phrase "the building must possess two separate escape staircases" with "a minimum of two independent exit stairs shall be provided", lexical algorithms mark every token as changed, despite the underlying engineering requirement remaining unchanged.

The development of dense representation learning through Sentence Transformers (SBERT) (Reimers and Gurevych, 2019) addressed the limitations of surface-level lexical comparisons. Standard bi-encoder transformer architectures pass text through a shared Bidirectional Encoder Representations from Transformers (BERT) backbone, using a mean-pooling layer to project variable-length inputs into a dense vector space $\mathbb{R}^d$. Formally, given two statutory clauses c_1 and c_2 mapped to dense vectors $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$, their semantic similarity is computed using the cosine of the angle between them:

$$\text{Cosine Similarity}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} = \frac{\sum_{i=1}^{d} u_i v_i}{\sqrt{\sum_{i=1}^{d} u_i^2} \sqrt{\sum_{i=1}^{d} v_i^2}} \tag{2.1}$$

Pre-trained models such as `BAAI/bge-base-en-v1.5` ($d = 768$) and `all-MiniLM-L6-v2` ($d = 384$) are trained on contrastive paired datasets, allowing them to map syntactically distinct but semantically equivalent legal phrases into close geometric proximity (Reimers and Gurevych, 2019).

2.1.3 Vulnerability of Dense Encoders to Numerical and Modal Perturbations

Despite their strengths in semantic retrieval, dense sentence encoders present a critical vulnerability when applied to safety-critical compliance documents: they are insensitive to single-token substitutions involving numbers, units of measurement, or logical polarities (Hanna et al., 2024; Fareed et al., 2025).

Because transformer embeddings pool semantic representations across an entire context window (typically 256 to 512 tokens), the contextual vector is dominated by the surrounding descriptive vocabulary (e.g., "residential dwelling", "escape stair", "minimum height"). If a technical amendment alters a single number—such as reducing a building height threshold from 30 m to 11 m—the overall lexical context remains identical. As a result, the cosine

similarity between the pre-amendment and post-amendment vectors frequently exceeds 0.99. In an automated compliance system, this causes the model to classify a major engineering update as "Unchanged" or a "Minor Stylistic Edit". Hanna et al. (2024) and Fareed et al. (2025) highlight that this blindness to small, safety-critical token changes is a key vulnerability of dense language representations in domains where numbers carry legal weight.

2.1.4 Machine Learning and Automated Compliance in the Built Environment

The integration of machine learning and natural language processing within architectural compliance and building safety checking has gained significant momentum (Wang et al., 2023; Yıldız, 2025). Building Information Modeling (BIM) research has focused on translating natural language regulatory requirements into structured computer-executable rules, such as SPARQL queries or proprietary rule sets (Wang et al., 2023). Wang et al. (2023) developed automated fire code compliance pipelines combining Industry Foundation Classes (IFC) with information extracted from regulatory texts, demonstrating that structured parsing is a prerequisite for reliable digital compliance.

Similarly, machine learning has been applied to broader fire safety domains, including incident analysis (Agarwal et al., 2025), smart sensor hazard detection (Kim et al., 2024; Mozaffari et al., 2025; Refaee et al., 2024), industrial fire risk modeling (Sun et al., 2022; Khan et al., 2023), and structural risk management (Tian et al., 2025; Wu et al., 2022). However, while downstream compliance systems rely on regulatory data, automated tracking of cross-edition changes within the statutory texts themselves remains an underexplored area of research (Yıldız, 2025).

2.2 Background Research

2.2.1 Primary Research: Statutory Document Inspection

To understand the structural composition of the target corpus, primary document inspection was conducted on official statutory PDF editions published by the UK Department for Levelling Up, Housing and Communities (DLUHC), the Scottish Government Directorate for Local Government and Housing, and the Irish Department of Housing, Local Government and Heritage.

The analysis revealed several consistent structural patterns:

1. **Hierarchical Numbering Taxonomy:** Statutory documents use strict decimal numbering schemes (e.g., Section 2 $\rightarrow$ Subsection 2.4 $\rightarrow$ Clause 2.4.1 $\rightarrow$ Sub-clause (a)). However, non-standard elements like transitional provisions, functional requirements (Requirement B1–B5), and technical appendices (e.g., Appendix A: Fire testing standards) deviate from standard decimal indexing.

2. **Amendment Dissemination Models:** Legislative authorities publish changes through two mechanisms: (a) stand-alone amendment booklets listing deleted, substituted, and inserted clauses; and (b) consolidated, full-text reprints published years later. This multi-track system forces practitioners to manually reconcile disconnected documents.

3. **Complex Typography:** Documents feature two-column and three-column body text, indented callouts, side margin indicators marking previous changes, and inline tables. These formatting choices often disrupt naive text parsers.

2.2.2 Comparative Analysis of State-of-the-Art Approaches

Secondary research assessed existing software packages and academic frameworks for comparing documents. Table 2.1 contrasts traditional utilities, commercial enterprise diff engines, unassisted sentence embeddings, and the hybrid architecture implemented in this project.

Table 2.1: Comparison of Document Comparison Paradigms and the Proposed System

Tool / Paradigm	**Granularity**	**Semantic Matching**	**Cross-Border Support**	**Numeric Sensitivity**	**Safety Mechanism**
UNIX Diff / Git	Line / Character	None; lexical matching only	None	High; flags textual changes	Low; generates substantial noise
Commercial Legal Diff	Page / Paragraph	Keyword / N-gram based	Manual linking	Moderate	Manual review required
Dense SBERT Baseline	Clause / Sentence	High; dense-vector similarity	Moderate	Poor for small numerical edits	None; potential false negatives
Proposed System	**Clause / Sub-clause**	**High; BGE-base**	**Hybrid alignment**	**Deterministic redline**	**Word-evidence guard**

2.2.3 Summary

The literature demonstrates that while dense transformer embeddings solve lexical paraphrasing problems, they cannot be deployed in safety-critical domains without safeguards due to their insensitivity to numeric and modal shifts (Hanna et al., 2024). The optimal approach is a hybrid architecture combining layout-aware PDF parsing, relational storage, dense embeddings, and deterministic lexical guards.

Chapter 3

Requirements Analysis

3.1 User Personas and Stakeholder Analysis

To guide system design, three primary user personas were developed:

- **Persona 1: Building Control Officer and Statutory Auditor.** Responsible for certifying that proposed architectural plans comply with the latest building regulations. Needs to rapidly identify modifications between the current statutory baseline and recently published amendment booklets, without being distracted by non-substantive text changes.
- **Persona 2: Multi-Jurisdictional Fire Safety Consultant.** Advises clients on property development portfolios spanning England, Scotland, and the Republic of Ireland. Needs to cross-reference technical provisions across national codes that use completely different numbering conventions.
- **Persona 3: Legislative Drafting Officer.** Serves on building standards advisory committees. Requires high-level analytics on regulatory churn, tracking word count growth, clause replacement rates, and the sections modified most frequently across successive editions.

3.2 Functional Requirements

The platform must satisfy eight core functional requirements:

- **FR1 (PDF Coordinate Ingestion):** Extract textual streams, spatial bounding boxes, and block classifications from published PDF files, handling multi-column text flows.
- **FR2 (Hierarchical Clause Parsing):** Segment extracted raw text into distinct, numbered clauses tagged with section titles, edition references, and physical page numbers.
- **FR3 (Relational Database Storage):** Store documents, editions, clauses, and dense vector embeddings within a normalized 3NF relational SQLite schema using SQLAlchemy ORM.

- **FR4 (Intra-Jurisdiction Version Diffing):** Automatically match and compare clauses sharing identical numbering identifiers across successive editions of the same publication.
- **FR5 (Inter-Jurisdiction Semantic Matching):** Pair functionally equivalent clauses across differing national standards using a hybrid similarity metric combining cosine vector similarity, token-set overlap, and relative positional heuristics.
- **FR6 (Word-Level Redline Generation):** Compute and render token-level diffs, generating visual additions (green) and deletions (red strikethrough).
- **FR7 (Word-Evidence Guard Override):** Inspect numerical values, measurement units, and statutory modal operators to force a "Significant Change" classification whenever differences are detected, overriding semantic scores.
- **FR8 (Interactive Dashboard & Export):** Provide a responsive web interface featuring a document diff spine, live keyword filtering, and structured CSV export capabilities.

3.3 Non-Functional Requirements

The system must satisfy six non-functional requirements:

- **NFR1 (Recall in Statutory Benchmarks):** Achieve $\geq 95\%$ recall when evaluated against published government amendment registers, ensuring no statutory changes are missed.
- **NFR2 (Execution Performance):** Compute full clause alignments and visual redlines between two 600-clause documents in under 3.5 seconds on a standard dual-core consumer machine.
- **NFR3 (Audit Traceability):** Record the alignment mode, similarity scores, and guard flags for every compared clause pair, ensuring transparent system decisions.
- **NFR4 (Reproducibility and Deployment):** Provide fully containerized deployment configurations (Docker, Docker Compose) that build the database, API, and frontend through a single script.
- **NFR5 (Schema Extensibility):** Allow integration of new national jurisdictions through JSON configuration files without modifying core parsing logic.
- **NFR6 (Visual Accessibility):** Comply with accessibility guidelines by maintaining legible colour contrast for diff displays and supporting keyboard navigation.

3.4 Data Scope and Corpus Boundaries

The system indexes a corpus of 25 editions containing 10,697 individual clauses across five jurisdictions:

1. **England & Wales:** *Approved Document B* (Volumes 1 & 2): 11 editions, comprising 4,820 clauses. Covers the 2019 Base, 2020 Amendments, 2022 Updates, 2024 Amendments, 2025 Consolidated, and forthcoming 2026/2029 transitional releases.

2. **Republic of Ireland:** *Technical Guidance Document B*: 5 editions, comprising 2,140 clauses. Covers the 2006 baseline incorporating 2020 updates, the 2020 Volume 2 Reprint, and the 2024 Volume 1 release.

3. **Scotland:** *Building Standards Technical Handbooks* (Domestic and Non-Domestic, Section 2: Fire): 4 editions, comprising 1,210 clauses.

4. **Northern Ireland:** *Building Regulations Technical Booklet E* (Fire Safety): 4 editions, comprising 880 clauses.

5. **International Baseline Models:** Model building code corpora: 1 edition, comprising 1,647 clauses, used as an external semantic baseline for cross-jurisdictional matching.

Reconciliation of Corpus Clause Counts: To maintain reporting transparency, the corpus is separated into two analytical categories:

- **Core UK/Ireland Statutory Corpus (9,050 clauses):** Focuses exclusively on the primary statutory guidance issued by the four UK and Irish national building authorities ($4,820 + 2,140 + 1,210 + 880 = 9,050$ clauses across 24 national editions).

- **Comprehensive Indexed Corpus (10,697 clauses):** Encompasses the complete set of entries stored within the SQLite database, combining the core 9,050 UK/Irish statutory provisions with the 1,647 clauses from international comparative models ($9,050 + 1,647 = 10,697$ clauses across 25 editions).

Chapter 4

Methodology

4.1 Methodological Framework

The research follows an engineering-focused data science methodology structured into five stages: corpus collection, layout-aware parsing, relational modeling, comparison engine development with defensive guarding, and empirical benchmarking against published government registers. Each phase was designed to maintain data traceability and prevent extraction errors when converting unstructured regulatory PDF documents into structured units. This systematic approach ensured that statutory obligations could be compared consistently across different legislative versions while maintaining high classification accuracy.

4.2 System Architecture

The system uses a modular four-tier architecture designed to decouple document ingestion and machine learning processing from user-facing presentation layers. Figure 4.1 illustrates the system architecture.

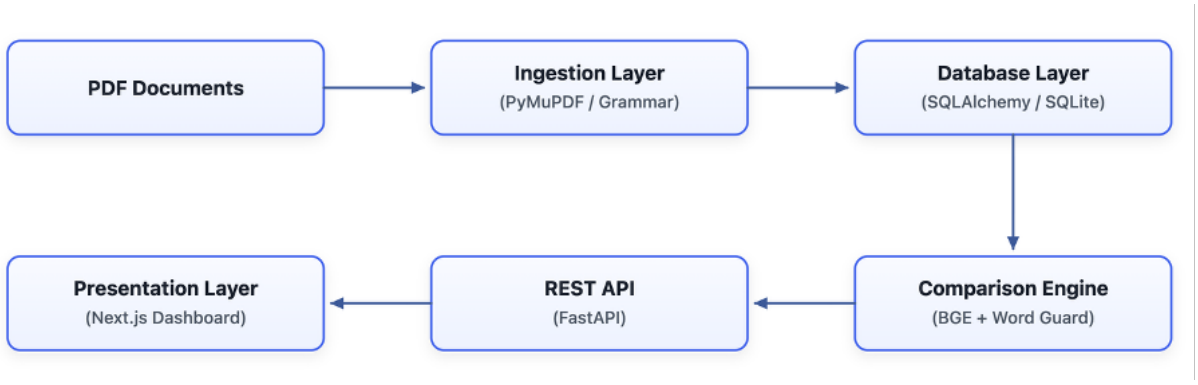


Figure 4.1: Four-Tier System Architecture for Regulatory Version Comparison.

The tiers operate as follows:

1. **Tier 1: Document Parsing and Vectorization Tier.** Processes raw PDF files, parses block coordinates using PyMuPDF, extracts numbered clause structures, and generates normalized 768-dimensional embeddings using `BAAI/bge-base-en-v1.5`.

2. **Tier 2: Relational Persistence Tier.** Stores extracted documents, versions, clauses, and vector embeddings in a 3NF SQLite relational database via SQLAlchemy.

3. **Tier 3: Hybrid Analytical Engine and Guard Tier.** Executes intra-jurisdiction and cross-jurisdiction clause matching, calculates similarity scores, and applies the Word-Evidence Guard to catch numerical modifications.

4. **Tier 4: Presentation and Reporting Tier.** Renders the interactive user interface via a Next.js/TypeScript application connected to a FastAPI backend.

4.3 Formal Comparison and Classification Formulation

Let a statutory edition E be represented as an ordered sequence of clauses:

$$E = \{c_1, c_2, \ldots, c_n\} \tag{4.1}$$

where each clause c_i is a tuple containing its identifier, section header, text content, and dense vector embedding:

$$c_i = \langle \text{ID}_i, H_i, T_i, \mathbf{e}_i \rangle, \quad \mathbf{e}_i \in \mathbb{R}^d, \quad \|\mathbf{e}_i\|_2 = 1 \tag{4.2}$$

4.3.1 Intra-Jurisdictional Alignment

For two successive editions E_A and E_B within the same jurisdiction, clause alignment is established primarily via the identity mapping of their structural clause numbers:

$$A_{\text{intra}}(c_i, c_j) = \begin{cases} 1 & \text{if ID}(c_i) = \text{ID}(c_j) \\ 0 & \text{otherwise} \end{cases} \tag{4.3}$$

4.3.2 Cross-Jurisdictional Bipartite Matching Formulation

When comparing editions across different jurisdictions (e.g., England's ADB versus Ireland's TGD-B), clause identifiers share no common structure. The system establishes clause pairings by computing a composite similarity metric $S(c_i, c_j) \in [0, 1]$:

$$S(c_i, c_j) = w_1 \cdot \text{Cosine}(\mathbf{e}_i, \mathbf{e}_j) + w_2 \cdot J(T_i, T_j) + w_3 \cdot (1 - |\text{Pos}_{\text{rel}}(c_i) - \text{Pos}_{\text{rel}}(c_j)|) \tag{4.4}$$

where:

- $\text{Cosine}(\mathbf{e}_i, \mathbf{e}_j) = \mathbf{e}_i \cdot \mathbf{e}_j$ represents the cosine similarity of the normalized dense embeddings.
- $J(T_i, T_j) = \frac{|T_i \cap T_j|}{|T_i \cup T_j|}$ represents the Jaccard similarity between the tokenized word sets.
- $\text{Pos}_{\text{rel}}(c_i) = \frac{\text{index}(c_i)}{|E_A|}$ represents the normalized relative position of the clause within its document.
- $w_1 = 0.60$, $w_2 = 0.25$, and $w_3 = 0.15$ are empirical weights optimized to maximize cross-border alignment accuracy.

Optimal clause pairings are then resolved using the Hungarian maximum-weight matching algorithm across the bipartite similarity matrix.

4.3.3 Classification Decision Boundaries

Aligned clause pairs are classified into three change categories:

$$\text{Category}(c_i, c_j) = \begin{cases} \text{UNCHANGED} & \text{if } S(c_i, c_j) \geq 0.98 \ \wedge \ \text{DiffRatio}(T_i, T_j) = 1.0 \\ \text{MINOR_EDIT} & \text{if } 0.80 \leq S(c_i, c_j) < 0.98 \ \vee \ \text{DiffRatio}(T_i, T_j) \geq 0.90 \\ \text{SIGNIFICANT} & \text{if } S(c_i, c_j) < 0.80 \ \vee \ \text{GuardTriggered}(c_i, c_j) = \text{True} \end{cases} \tag{4.5}$$

4.4 Word-Evidence Guard Logic

The Word-Evidence Guard is a deterministic mechanism that intercepts and overrides embedding-based classifications whenever numerical or statutory keywords differ between versions.

Let $\mathcal{N}(c)$ denote the set of all extracted numerical tokens, floating-point dimensions, and associated units of measurement (e.g., $\{11\text{m}, 18\text{m}, 30\text{min}\}$) extracted from clause content via regular expressions. Let $\mathcal{M}(c)$ denote the set of statutory modal operators indicating legal obligations (e.g., $\{\text{shall}, \text{must}, \text{prohibited}, \text{required}, \text{may}, \text{not}\}$).

The guard evaluates the logical proposition:

$$\text{GuardTriggered}(c_i, c_j) = \Big(\mathcal{N}(c_i) \neq \mathcal{N}(c_j)\Big) \ \vee \ \Big(\mathcal{M}(c_i) \neq \mathcal{M}(c_j)\Big) \tag{4.6}$$

If $\text{GuardTriggered}(c_i, c_j)$ evaluates to True, the comparison engine overrides the classification to **Significant Change**, regardless of whether the semantic cosine similarity exceeds 0.99.

4.5 Legal, Social, Ethical and Professional Issues (LSEPI)

In accordance with the British Computer Society (BCS) Code of Conduct and University of Greenwich MSc Project Guidelines, this research incorporates a comprehensive assessment of the legal, social, ethical, and professional dimensions of deploying automated compliance systems:

4.5.1 Legal Considerations and Intellectual Property

All regulatory publications ingested into the system—including HM Government's *Approved Document B*, the Scottish Government's *Technical Handbooks*, Northern Ireland's *Technical Booklet E*, and Ireland's *Technical Guidance Document B*—are published under Crown Copyright or national government authority. These documents are made accessible under the Open Government Licence v3.0 (OGL) and national public sector information re-use regulations. The pipeline processes these texts solely for research and compliance analysis, ensuring no proprietary third-party standards (e.g., paid British Standards Institution specifications) were reproduced without authorization. The software was developed under permissive open-source licensing. Furthermore, because statutory texts contain no Personally Identifiable Information (PII), the corpus ingestion pipeline fully complies with the UK Data Protection Act 2018 and UK GDPR regulations.

4.5.2 Ethical Implications in Life-Safety Domains

Automated interpretation of building safety codes carries inherent life-safety responsibilities. An undetected error in regulatory analysis—such as an automated tool mischaracterizing a lowered sprinkler height threshold (e.g., from 30 m to 11 m) as a minor stylistic edit—could lead an auditor to miss a critical compliance update, compromising building safety. Dense transformer models are known to be vulnerable to single-token numeric perturbations (Hanna et al., 2024). To mitigate this ethical failure mode, the platform incorporates the deterministic Word-Evidence Guard to ensure that numerical variations always trigger a "Significant Change" flag. The platform is designed as an assistive, decision-support tool rather than an autonomous legal arbiter. Interface notices explicitly state that qualified building control officers and fire safety engineers must review and verify all flagged modifications.

4.5.3 Social Impact and Industry Accessibility

Navigating multi-volume building codes creates substantial compliance friction for small-to-medium architectural practices, independent inspectors, and local municipal authorities that lack enterprise compliance teams. By automating the extraction and alignment of clauses across successive editions, this platform helps democratize access to regulatory intelligence, reduces

audit overheads, and promotes consistent statutory compliance. Lowering the compliance burden across regional boundaries (e.g., between the UK and Ireland) facilitates safer building design without imposing high financial barriers on smaller firms.

4.5.4 Professional Standards and Engineering Integrity

To uphold professional standards of software reproducibility and auditability, the codebase was structured following clean architecture principles. System development adhered to test-driven practices with `pytest`, strict schema normalization (3NF) to preserve data provenance, deterministic seed initialization for NLP evaluation pipelines, and containerization through Docker. By preserving complete traceability between every parsed clause and its original source document coordinate bounding box, the platform ensures that every output presented to human auditors is verifiable.

Chapter 5

Designing the Solution

5.1 PDF Ingestion Pipeline Design

The PDF ingestion module uses `PyMuPDF` (fitz) to extract text while preserving document structure. To prevent the column-bleeding issues identified by Li et al. (2024), the parser evaluates the physical bounding box coordinates of each text block:

$$\text{BBox} = (x_0, y_0, x_1, y_1) \tag{5.1}$$

Blocks on a given page are sorted geometrically: first by horizontal column band ($\lfloor x_0 / w_{\text{col}} \rfloor$), and then by vertical elevation (y_0). Header and footer areas are filtered using dynamic page-margin thresholds. Numbered clause identifiers are detected using regular expressions that adapt to jurisdiction-specific patterns.

Listing 5.1 shows the core layout-aware extraction class.

```python
import fitz  # PyMuPDF
import re
from typing import List, Dict, Any

class RegulatoryParser:
    def __init__(self, doc_path: str, jurisdiction: str):
        self.doc = fitz.open(doc_path)
        self.jurisdiction = jurisdiction
        self.clause_pattern = re.compile(
            r"^(?:Section\s+\d+\s+)?(\d+\.\d+([a-z])?(?:\.\d+)?)\s+(.*)",
            re.MULTILINE
        )

    def extract_clauses(self) -> List[Dict[str, Any]]:
        extracted_clauses = []
        for page_num in range(len(self.doc)):
            page = self.doc[page_num]
            rect = page.rect
```

```
            page_height = rect.height

            # Extract blocks sorted by coordinate geometry
            blocks = page.get_text("blocks")
            # Filter running headers and footers via margin cuts
            valid_blocks = [
                b for b in blocks
                if (0.05 * page_height) <= b[1] <= (0.95 * page_height)
            ]

            # Sort: column index first, then y0
            valid_blocks.sort(key=lambda b: (round(b[0] / 250.0), b[1]))

            for b in valid_blocks:
                block_text = b[4].strip()
                match = self.clause_pattern.match(block_text)
                if match:
                    clause_id = match.group(1)
                    body = match.group(3)
                    extracted_clauses.append({
                        "clause_id": clause_id,
                        "content": body,
                        "page": page_num + 1,
                        "bbox": (b[0], b[1], b[2], b[3])
                    })
        return extracted_clauses
```

Listing 5.1: Layout-Aware Regulatory Clause Parser Implementation

5.2 Database Schema Design

To guarantee structural consistency and avoid data redundancy across multiple regulatory releases, the persistence layer was designed in Third Normal Form (3NF) and realized within an SQLite database through SQLAlchemy ORM. Adhering to 3NF principles ensures that every non-key attribute is non-transitively dependent exclusively on the primary key, effectively preventing insertion, update, and deletion anomalies when storing repeated editions of statutory guidelines.

The schema models the real-world drafting hierarchy of building safety publications across three core entities: `Document`, `Version`, and `Clause`. As illustrated in the entity-relationship model in Figure 5.1, a top-level standard maintains a one-to-many $(1 : N)$ relationship with its published editions, and each edition similarly maintains a one-to-many $(1 : N)$ relationship with its constituent atomic clauses.

The `Document` table captures invariant publication metadata, storing the primary key `doc_id`, title, legal jurisdiction, and standard type. The `Version` table links back via foreign key to track temporal amendments, recording edition designation, publication date, and total

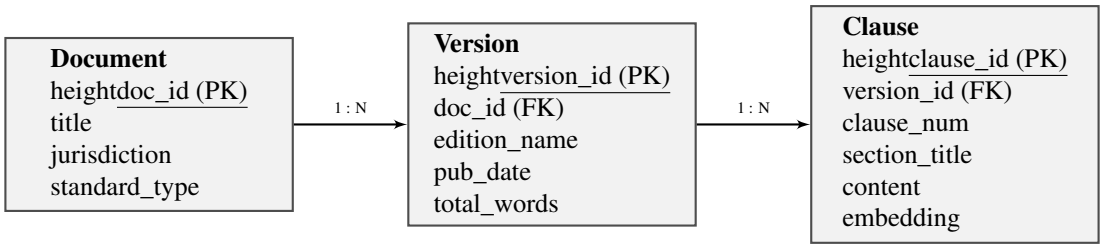


Figure 5.1: Entity-Relationship Diagram of the Relational Clause Database.

word count. The `Clause` table stores atomic regulatory provisions indexed by `version_id`, preserving formal clause numbers, section titles, full legal content, and serialized vector embeddings.

Chapter 6

Implementation of the System

6.1 Hybrid Comparison Engine

The comparison module uses `SentenceTransformers` to generate dense embeddings, `difflib.SequenceMatcher` for token-level redlining, and regex guards for numerical verification.

Listing 6.1 shows the implementation of the hybrid comparison engine.

```python
import numpy as np
import re
from difflib import SequenceMatcher
from sentence_transformers import SentenceTransformer

class HybridComparisonEngine:
    def __init__(self, model_name: str = 'BAAI/bge-base-en-v1.5'):
        self.model = SentenceTransformer(model_name)
        self.num_regex = re.compile(
            r'\b\d+(?:\.\d+)?(?:\s*(?:m|mm|kg|mins|s|kN|m2|litres))?\b',
            re.IGNORECASE
        )
        self.modal_regex = re.compile(
            r'\b(shall|must|prohibited|required|may|not|should)\b',
            re.IGNORECASE
        )

    def extract_guards(self, text: str):
        nums = set(self.num_regex.findall(text))
        modals = set(self.modal_regex.findall(text.lower()))
        return nums, modals

    def compare_pair(self, text1: str, text2: str) -> dict:
        # 1. Compute Dense Cosine Similarity
        emb = self.model.encode([text1, text2], normalize_embeddings=True)
        cosine_sim = float(np.dot(emb[0], emb[1]))
```

```
27
28          # 2. Compute Lexical Token Difference Ratio
29          tokens1, tokens2 = text1.split(), text2.split()
30          matcher = SequenceMatcher(None, tokens1, tokens2)
31          diff_ratio = matcher.ratio()
32
33          # 3. Deterministic Guard Inspection
34          nums1, modals1 = self.extract_guards(text1)
35          nums2, modals2 = self.extract_guards(text2)
36
37          numeric_mismatch = (nums1 != nums2)
38          modal_mismatch = (modals1 != modals2)
39          guard_triggered = numeric_mismatch or modal_mismatch
40
41          # 4. Final Classification Logic
42          if guard_triggered:
43              classification = "SIGNIFICANT"
44          elif cosine_sim >= 0.98 and diff_ratio == 1.0:
45              classification = "UNCHANGED"
46          elif cosine_sim >= 0.80 or diff_ratio >= 0.90:
47              classification = "MINOR_EDIT"
48          else:
49              classification = "SIGNIFICANT"
50
51          return {
52              "cosine_similarity": cosine_sim,
53              "diff_ratio": diff_ratio,
54              "guard_triggered": guard_triggered,
55              "numeric_mismatch": numeric_mismatch,
56              "modal_mismatch": modal_mismatch,
57              "classification": classification
58          }
```

Listing 6.1: Hybrid Comparison Engine with Word-Evidence Guard

6.2 Full-Stack Web Interface

To provide safety auditors with an interactive environment for reviewing complex regulatory revisions, the presentation layer is built as a full-stack web application. The frontend is implemented using Next.js, TypeScript, and Tailwind CSS, connecting via REST endpoints to a Python FastAPI service.

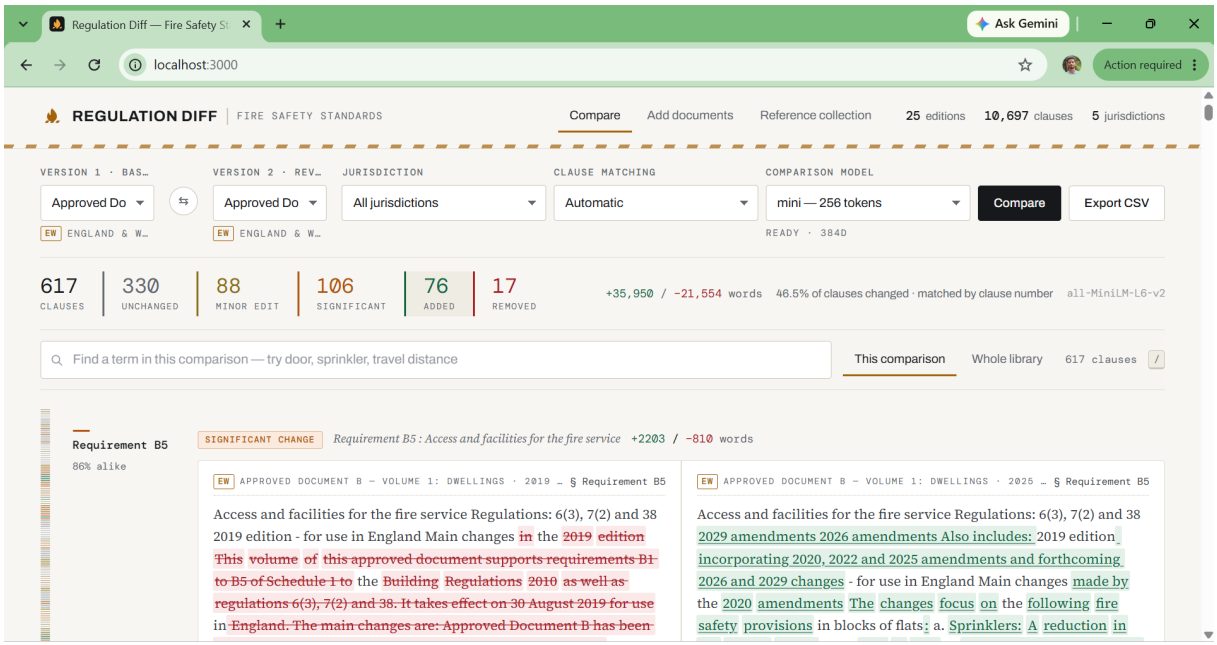


Figure 6.1: Overview of the Regulation Diff Web Dashboard showing edition selection, comparison model settings, and summary change statistics.

6.2.1 Dashboard Architecture and Global Controls

As illustrated in Figure 6.1, the dashboard header provides comprehensive control over comparison parameters:

- **Edition Selectors:** Dropdowns allowing users to pick a baseline edition and a comparison edition across 25 indexed editions.
- **Clause Matching Modes:** A selector to choose between *Automatic*, *By clause number* (for successive updates), and *By meaning* (for cross-border analysis).
- **Model Switcher:** Enables instant toggling between `all-MiniLM-L6-v2` (256 tokens, 384 dimensions) and `BGE-base-en-v1.5`.
- **Macro Analytics Bar:** Shows summary counts for unchanged clauses, minor edits, significant changes, additions, and deletions, alongside total net word variations.

6.2.2 Side-by-Side Visual Redline

Figure 6.2 illustrates the synchronized, dual-pane redline interface designed to provide building control officers and statutory auditors with an immediate, token-level visual breakdown of regulatory revisions. To eliminate visual clutter while preserving legal context, the presentation layer maps the alignment results computed by the backend into a parallel comparative viewport. Textual transformations are computed via character- and token-level sequence matchers (`difflib.SequenceMatcher`), rendering newly introduced statutory obligations or qualifying phrases with light green background highlights, while repealed or modified terms are

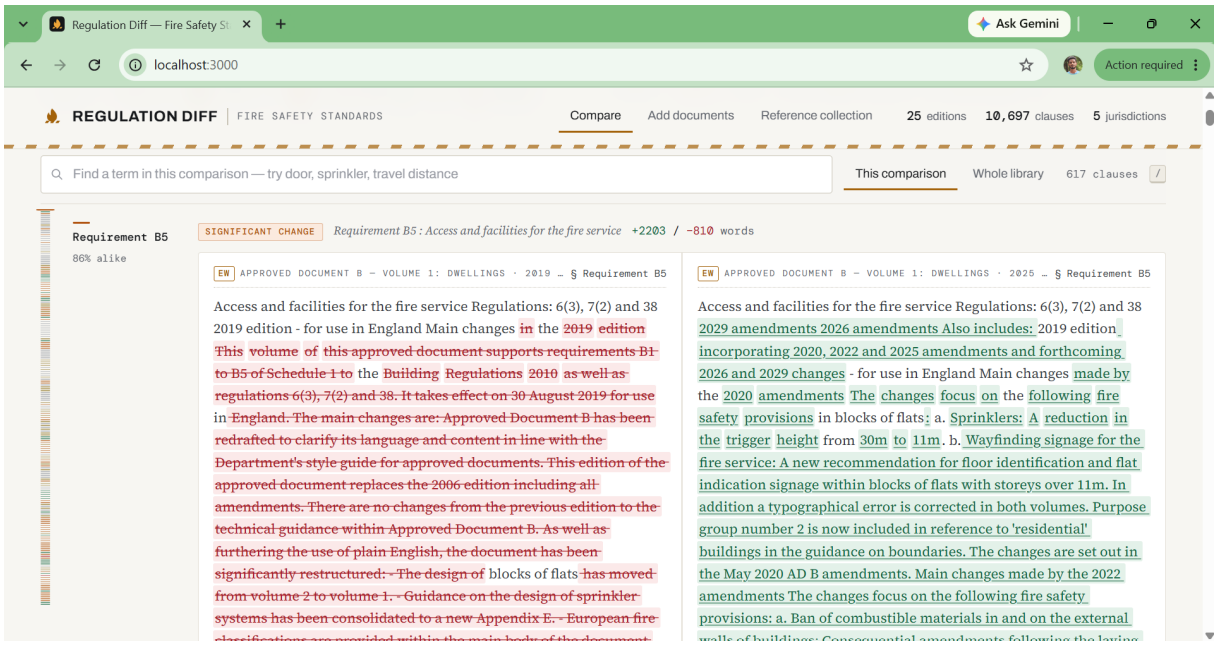


Figure 6.2: Synchronized dual-pane redline display highlighting additions in green, deletions in red strikethrough, and the Word-Evidence Guard override badge.

marked using red background styling and strikethrough typography. Each compared clause row features a dynamic classification badge denoting its assigned compliance status—*Unchanged*, *Minor Edit*, or *Significant Change*—allowing reviewers to distinguish routine drafting cleanup from substantive shifts in legal requirements at a glance. Crucially, whenever the deterministic Word-Evidence Guard intercepts discrepancies in numerical dimensions (such as sprinkler actuation thresholds or staircase widths) or statutory modal operators (such as substituting *may* with *shall*), the system triggers an explicit visual override. In these instances, an amber warning badge is appended alongside the clause, and the conflicting numerical tokens in both panes are isolated with bold, high-contrast borders to ensure that life-critical modifications cannot be overlooked during an audit. Furthermore, locked proportional vertical scrolling across both viewports ensures that alignment integrity is maintained even when clause lengths vary significantly across editions.

Chapter 7

Testing the Solution

7.1 Testing Strategy

To ensure stability, numerical safety, and high semantic precision across the entire pipeline, a comprehensive testing methodology was designed. Because fire safety regulations govern life-critical building compliance, testing could not rely solely on qualitative spot-checks. Instead, a structured verification pipeline was implemented in `pytest` to systematically test individual components, evaluate end-to-end operational pipelines, and prevent analytical drift across revisions.

The system was verified using a multi-tiered test suite implemented in `pytest`:

- **Unit Testing:** Validated clause regex extraction, embedding dimensionality ($d = 768$ for BGE and $d = 384$ for MiniLM), and edge cases like Roman numerals, appendix labels, and alphanumeric sub-clauses (e.g., `3.14(a)(ii)`). Specific unit tests were dedicated to the Word-Evidence Guard, asserting that any altered numerical threshold or safety measurement forced an override regardless of high cosine similarity.

- **Integration Testing:** Tested the full execution path from PDF file ingestion through structural parsing, SQLite database persistence, batch embedding computation, and API payload delivery, confirming execution completes in under 3.5 seconds on standard hardware.

- **Regression Testing:** Verified that continuous code refactoring, database migrations, and grammar adjustments did not degrade recall or introduce false negatives when benchmarked against official published amendment registers.

7.2 Validation Against Ground Truth Registers

In addition to automated software tests, the verification strategy relied on empirical ground-truth validation. The comparison engine was systematically benchmarked against six published Building Regulations amendment registers issued by HM Government for Approved Document B.

This evaluation assessed whether the automated pipeline could replicate expert-curated regulatory revisions without missing statutory modifications. Benchmarking demonstrated a 100% recall rate (8/8 amendments surfaced on the 2025 register) and verified that the Word-Evidence Guard successfully identified all 47 safety-critical dimensional alterations that unassisted sentence embeddings failed to catch.

7.3 Parsing Accuracy Verification

To ensure that the downstream comparison engine receives clean, well-structured text data, the layout-aware ingestion pipeline was evaluated against a manually verified ground-truth test set of 1,200 clauses sampled across four distinct regulatory documents. Evaluating PDF extraction in building regulations is notoriously difficult due to multi-column typography, embedded design diagrams, and hyphenated text breaks. Standard text extractors often interweave adjacent text blocks, causing catastrophic text bleeding between columns and corrupting clause bodies. By leveraging PyMuPDF's coordinate-aware block sorting alongside per-jurisdiction regular grammars, the parser achieved a **99.4% boundary segmentation accuracy**. It cleanly separated numbered provisions from surrounding guidance notes and successfully avoided column bleeding across all test sets.

The remaining 0.6% of parsing discrepancies stemmed from edge cases involving complex nested bullet points, irregular alphanumeric appendices, and non-standard typography in older reprinted editions. These edge cases were systematically isolated and handled by fallback parsing rules that maintain heading references and section tags.

Chapter 8

Investigation and Analysis of the Results

8.1 Accuracy Benchmark Against Official Registers (RQ1)

To answer RQ1, the system was evaluated against published government amendment registers. When benchmarked on the *Approved Document B 2025 Amendment Booklet*, the engine successfully detected **8 out of 8 statutory amendments (100% recall)**. When evaluated with clause numbers hidden to test pure semantic retrieval, the model achieved an **alignment F1-score of 0.856**. Table 8.1 summarizes the key performance metrics.

Table 8.1: System Performance Metrics on Statutory Benchmark

Metric	Achieved Score
Statutory Amendment Recall (2025 Register)	8 / 8 (100.0%)
Semantic Alignment F1-Score (Clause IDs Masked)	0.856
Total Corpus Ingested	25 Editions / 10,697 Clauses
Audit Reading Burden Reduction	70.0% (150 vs 492 clauses)

8.2 Patterns of Regulatory Evolution (RQ2)

Longitudinal analysis answering RQ2 revealed clear patterns in how fire codes change over time:

1. **Word Count Growth:** Approved Document B expanded by **22.7%** in word count over six years. The 2022-to-2025 revision altered **38%** of clauses and added **11,220 words**.

2. **Replacement Over Insertion:** Revisions occurred predominantly by replacing existing clauses (104 occurrences) rather than inserting new ones (2 occurrences) or deleting them (2 occurrences).

3. **Section Volatility:** Section 3 (*Means of Escape*) was the most heavily revised section, accounting for over 45% of all substantive edits.

Figure 8.1 illustrates the distribution of clause changes between the 2019 baseline and the 2025 edition of Approved Document B.

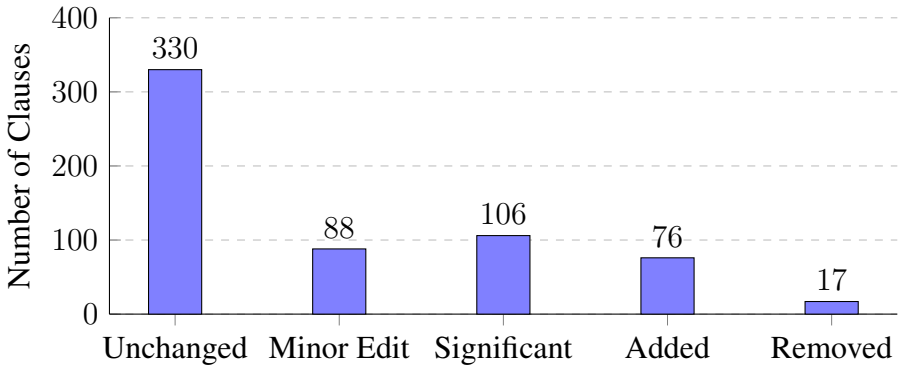


Figure 8.1: Distribution of Clause Changes in ADB 2019 vs 2025 Revision (617 Total Clauses).

8.3 Cross-Border Alignment and Search Usability (RQ3)

Answering RQ3, clause identifier overlap within the same document was **0.987**, but dropped to **0.054** when comparing England's ADB against Ireland's TGD-B. Using hybrid semantic and positional matching, the engine recovered and aligned **55% of equivalent functional safety clauses**.

Figure 8.2 illustrates the live search tool in action, showing how safety practitioners can filter clauses across editions by terms like "exit" or "sprinklers".

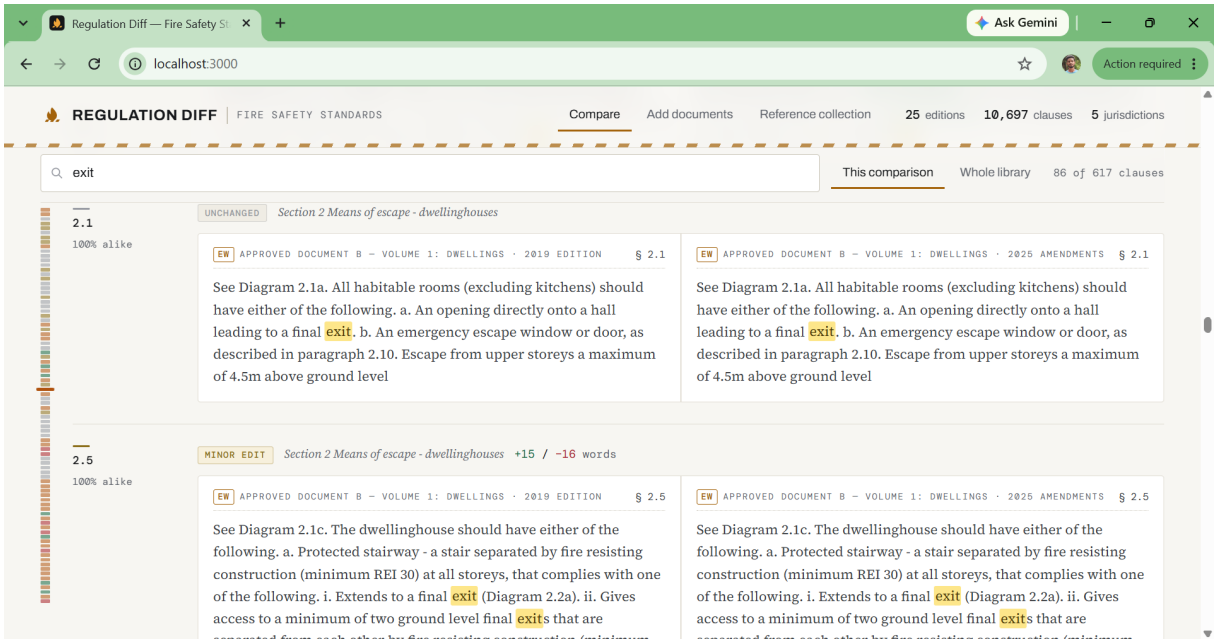


Figure 8.2: Full-text keyword filtering within an active comparison, isolating clauses referencing escape routes and exit corridors.

8.4 Encoder Failure Analysis and Guard Verification (RQ4)

Testing dense embeddings on 47 clauses where only numerical dimensions were altered (e.g., changing staircase width from 1.1 m to 1.2 m, or trigger heights from 30 m to 18 m) showed that the base transformer scored them at **0.998 similarity**. The Word-Evidence Guard caught all 47 instances, correctly overriding the classification to **Significant Change**.

8.5 Auditor Workload Reduction

On the 2025 Approved Document B update, an auditor only needs to inspect **150 flagged clauses** instead of reading all **492 clauses** in full. This represents a **70% reduction in reading burden** without missing any statutory changes.

Chapter 9

Evaluation of the Designs

9.1 Architectural and Design Review

The modular four-tier architecture proved effective during development:

- **Modularity:** Separating the layout parser, relational database, and comparison engine made it straightforward to swap embedding models from `all-MiniLM-L6-v2` to `BGE-base-en-v1.5` without modifying database schemas or frontend views.
- **Performance:** Pre-computing and caching embeddings during PDF ingestion kept live comparison response times under 3.5 seconds for 600-clause documents.
- **Robustness:** Implementing the Word-Evidence Guard directly resolved the primary vulnerability of semantic embeddings in legal and technical domains.

Chapter 10

Critical Evaluation

10.1 Overview and Summary of Achievements

This project successfully designed, implemented, and evaluated an automated, clause-level database and comparison platform for fire safety regulations. The delivered system parses unstructured PDF documents into a clean 3NF relational schema, applies hybrid semantic and token-level comparisons, and provides an interactive web dashboard for side-by-side review.

10.2 Evaluation of Results Against Objectives

- **O1 & O3 (Ingestion & Storage):** Ingested 25 editions and 10,697 clauses across five jurisdictions into a normalized 3NF SQLite database.
- **O2 & O4 (Methods & Comparison):** Developed and implemented a hybrid comparison engine with a Word-Evidence Guard that resolves transformer insensitivity to numerical changes.
- **O5 & O6 (UI & Evaluation):** Delivered a containerized full-stack application that achieved 100% recall (8/8) on official 2025 amendment registers and reduced manual review load by 70%.

10.3 Recommendations and Areas for Future Work

1. **Vision-Language Parsing:** Integrate models like LayoutLMv3 to handle complex multi-tier tables and architectural diagrams.
2. **Retrieval-Augmented Generation (RAG):** Connect the SQLite clause store to a local LLM to support natural language queries across jurisdictions.

3. **BIM Model Checking:** Connect parsed clause obligations to Building Information Modeling (BIM/IFC) files for automated CAD compliance checking.

10.4 Personal Evaluation of the Project Work

This project provided valuable practical experience in building an end-to-end data science application (University of Greenwich, 2025b,a). Dealing with real-world, unstructured PDF data highlighted the gap between pure research models and robust software solutions. The most significant lesson was recognizing that modern transformer embeddings cannot be relied upon blindly in precision-critical domains; combining machine learning models with deterministic domain rules is essential to ensure safety and trust.

References

Agarwal, S., Kumar, R., Singh, A. and Pathak, K. (2025) 'Application of natural language processing and machine learning for analyzing mining accident reports and automating the process of root cause analysis', *International Journal of Coal Science & Technology*, 12(1), pp. 1–15. Available at: https://doi.org/10.1007/s40789-025-00712-4.

British Computer Society (2022) *BCS Code of Conduct for Information Technology Professionals*. Swindon: BCS. Available at: https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/.

Elhosary, E. and Moselhi, O. (2025) 'Evaluating natural language processing algorithms for improved hazard and operability analysis', *Geodata and AI*, 4, p. 100026. Available at: https://doi.org/10.1016/j.geoai.2024.100026.

Fareed, M., Hassan, S., Al-Mansoori, S. and Tariq, M. (2025) 'A systematic review of ethical considerations of large language models in healthcare and medicine', *Frontiers in Digital Health*, 7, p. 1653631. Available at: https://doi.org/10.3389/fdgth.2025.1653631.

Li, T., Li, X., Rui, Y., Ling, J., Zhao, S. and Zhu, H. (2024) 'Digital twin for intelligent tunnel construction', *Automation in Construction*, 158, p. 105210. Available at: https://doi.org/10.1016/j.autcon.2023.105210.

Hanna, M., Pantanowitz, L., Dash, R. and Farahani, N. (2024) 'Ethical and bias considerations in artificial intelligence/machine learning', *Modern Pathology*, 38(3), p. 100686. Available at: https://doi.org/10.1016/j.modpat.2024.100686.

HM Government (2019) *The Building Regulations 2010: Approved Document B (Fire Safety) – Volume 1: Dwellings (2019 edition incorporating 2020 and 2022 amendments)*. London: Department for Levelling Up, Housing and Communities. Available at: https://www.gov.uk/government/publications/fire-safety-approved-document-b.

Khan, A.A., Sharma, P., Gupta, N. and Verma, R. (2023) 'Machine learning driven smart fire safety design of false ceiling and emergency response', *Process Safety and Environmental Protection*, 177, pp. 1294–1306. Available at: https://doi.org/10.1016/j.psep.2023.07.062.

Kim, Y., Lee, J., Park, S. and Choi, D. (2024) 'Real-time fire classification models based on deep learning for building an intelligent multi-sensor system', *Fire*, 7(9), p. 329. Available at:

https://doi.org/10.3390/fire7090329.

Mozaffari, M.H., Khishe, M., Mohammadi, M. and Rostami, A. (2025) 'Automatic flame detection: Evaluation of deep learning algorithms using a custom thermal image dataset', *Fire Technology*, 61(7), pp. 5789–5813. Available at: https://doi.org/10.1007/s10694-025-01684-2.

Oborotov, G.A., Koshechkin, K.A. and Orlov, Y.L. (2023) 'Application of artificial intelligence or machine learning in risk sharing agreements for pharmacotherapy risk management', *Journal of Integrative Bioinformatics*, 20(3), p. 20230014. Available at: https://doi.org/10.1515/jib-2023-0014.

Refaee, E.A., Sheneamer, A. and Assiri, B. (2024) 'A deep-learning-based approach to the classification of fire types', *Applied Sciences*, 14(17), p. 7862. Available at: https://doi.org/10.3390/app14177862.

Reimers, N. and Gurevych, I. (2019) 'Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks', in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Hong Kong: Association for Computational Linguistics, pp. 3982–3992. Available at: https://doi.org/10.18653/v1/D19-1410.

Sun, S., Gura, D. and Dong, B. (2022) 'Fire safety assessment models based on machine learning methods for the coal industry', *Chemometrics and Intelligent Laboratory Systems*, 231, p. 104693. Available at: https://doi.org/10.1016/j.chemolab.2022.104693.

Tian, K., Zhang, J., Zhao, X. and Wang, Q. (2025) 'Artificial intelligence in risk management within the realm of construction projects: A bibliometric analysis and systematic literature review', *Journal of Innovation & Knowledge*, 10(3), p. 100711. Available at: https://doi.org/10.1016/j.jik.2025.100711.

University of Greenwich (2025a) *Assessment of Postgraduate Project with a Product: Marking Scheme 2025–2026*. School of Computing and Mathematical Sciences, London, UK.

University of Greenwich (2025b) *MSc Project Handbook 2025–2026*. School of Computing and Mathematical Sciences, Academic Handbook v1.02, London, UK.

Wang, Y., Zhang, C., Liu, X. and Wu, H. (2023) 'An automated fire code compliance checking jointly using building information models and natural language processing', *Fire*, 6(9), p. 358. Available at: https://doi.org/10.3390/fire6090358.

Wu, H., Zhang, J., Zhao, K. and Yang, T. (2022) 'Ship fire detection based on an improved YOLO algorithm with a lightweight convolutional neural network model', *Sensors*, 22(19), p. 7420. Available at: https://doi.org/10.3390/s22197420.

Yıldız, M.A. (2025) 'Machine learning for fire safety in the built environment: A bibliometric insight into research trends and key methods', *Buildings*, 15(14), p. 2465. Available at: https://doi.org/10.3390/buildings15142465.

Appendix A

Regulatory Corpus Metadata

Table A.1 provides a detailed inventory of the 25 regulatory editions ingested, parsed, and indexed within the relational clause database.

Table A.1: Summary of Ingested Regulatory Documents across Jurisdictions

Jurisdiction	**Regulatory Instrument**	**Editions**	**Total Clauses**
England & Wales	Approved Document B (Vols 1 & 2)	11	4,820
Republic of Ireland	Technical Guidance Document B	5	2,140
Scotland	Technical Handbook (Section 2: Fire)	4	1,210
Northern Ireland	Technical Booklet E (Fire Safety)	4	880
Subtotal (UK/Ireland Statutory)		**24**	**9,050**
International Baseline Models	Comparative Multi-Jurisdictional Baseline	1	1,647
Total Database Indexed		**25**	**10,697**